

Milestone 3
Haylee Zuba

Introduction

Project Objective

The objective of this project is to explore the relationship of taste in music and choice of video games. By considering song metadata and data from video game sales, we can get an idea of this relationship. My project is a recommendation engine that takes in a song or artist, and produces the video game with the highest similarity score. This score is calculated based on key features extracted from the datasets I used: year, emotional mappings, genre, and 1 more to be inserted later. To achieve this, I used multiple python libraries. I used pandas to manipulate the datasets, sklearn for normalization of data and creation/training of the machine learning models, matplotlib for creating visualizations of the data, and numpy for arithmetic manipulation of the data. For the UI, I used HTML, CSS, and Javascript to create the UI and add functionality. The data used in this project is the Audio Features from songs 1911-2011, Spotify Track Features, and video game sales trends, all available on Kaggle.

Type of Tool

For my Spring project, I have chosen to create a project that is exemplary of option 3: a recommendation engine. One of the most intriguing aspects of data science, to myself as a student, is training a machine learning model to recognize patterns and watch it learn to make its own decisions.

To create this model, I will need to utilize different python libraries. For the data analysis portion, pandas and numpy will be crucial to analyze the data. Once we ascertain the statistical summaries of the data and account for outliers and null values, we can use libraries such as matplotlib and seaborn for visualization of the data and store it using SQLite. After the data preprocessing and exploratory data analysis are performed, we can move on to constructing the machine learning model using Pytorch and Scikit libraries. These are the intended libraries for now, but there may crop up other libraries that will assist with functionality or function better for the intended use rather than the ones listed.

The total tech stack for this project is as follows:

- Backend: Python in Pycharm IDE, utilizing libraries such as Pandas, Numpy, Seaborn, Matplotlib, Pytorch, Scikit, and SQLite.
- UI: Javascript and Streamlit to create a functional and aesthetically pleasing UI.

Project Timeline

Since the due dates of the project have changed, I had to adjust my timetable accordingly in order to complete the project by the new due dates. Also, as the project has developed, I have had new ideas for features and functions that will create a fuller fleshed out project. The syllabus and course schedule note that 4/14 - 4/23 are designated for presentations, so the goal is to finish the project in its entirety before the presentations begin. Here is the timeline for Milestone 3:

Task	Date to be Completed	Estimated Time
Model Evaluation/Improvement	4/10	~6 hours
Model Interpretation	4/11	~4 hours

Bias identification and analysis	4/13	~4 hours
UI Finalization	4/15	~6 hours
Presentation and Final Report	4/16	~6 hours

EDA Recap

After performing EDA on the 3 datasets, a few conclusions were apparent. Since the DEAM dataset did not contain labels for song names, I ended up finding other datasets that made it easier to match songs with video games. After processing the datasets and preparing them for merging, I created the dataset used for the feature engineering. Since I have domain knowledge - meaning I can see the columns of the datasets and have a general idea of how the data will look, I chose features I thought most important in calculating similarity scores - popularity, and emotional level. If I had chosen better datasets earlier, I would have implemented a span feature that allowed items within 10 years of release to be considered. The Hidden Trends in Video Games dataset provides information on video games such as genre, release date, popularity globally and nationally, company, and platform of release. For purposes of this project, we don't need information on the publisher or the platform on which it is played. In order to prepare this dataset for feature engineering, I modified the dataset to only contain

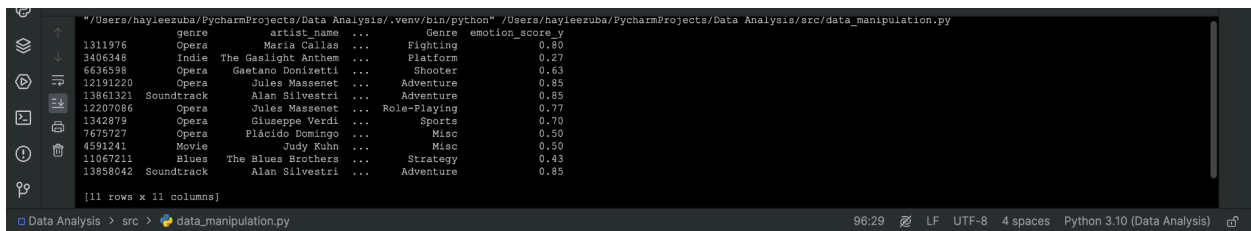
the columns I deemed necessary: Title, genre, Year of release, and ranking. These attributes are the most important because we can compare these to song metadata. From the Spotify track features, I created a subset containing the genre, artist_name, track_name, popularity, energy, tempo, and valence information. These features from the datasets that we explored in the EDA section are important for the feature engineering section of this project.

While the new Spotify dataset contained 18 columns of musical metadata, I wanted to round out the musical analysis in the code as much as possible. To supplement this, I also used the Audio Features of Songs from 1921 to 2011. This dataset is also a subset of the Million Song dataset. This dataset specifically focuses on the timbre of songs. Timbre refers to the tone color or quality of sounds. I included timbre in the emotional calculation because, as one of the eight pillars of music, it is key to determining the emotional coefficient of a song. For example, a user who prefers a song with a brighter timbre or sound quality might be well suited for an adventure game of similar timbre. On the other hand, a user who prefers songs with a dark timbre, would probably prefer to play a horror game. These considerations create a more nuanced song and game match.

Feature Engineering

Feature Creation

In order to train the 3 machine learning models on the data, it is crucial to manipulate the datasets into an aggregated dataset. For this project specifically, I merged the Video Game Sales, Spotify Track Features, and Audio Features datasets into one dataset. Then, I will be able to properly split the data into training, testing, and validation sets. I want the machine learning models to predict a video game based on a song, so after performing EDA on the datasets used I can determine which features to create from the datasets. Below shows the final dataset with some original columns from the datasets and some new features I created.



```
"/Users/hayleezuba/PycharmProjects/Data Analysis/.venv/bin/python" /Users/hayleezuba/PycharmProjects/Data Analysis/src/data_manipulation.py
```

	Genre	artist_name	...	Genre	emotion_score_y
1311976	Opera	Maria Callas	...	Fighting	0.80
3406348	Indie	The Gaslight Anthem	...	Platform	0.27
6636598	Opera	Gaetano Donizetti	...	Shooter	0.63
12191220	Opera	Jules Massenet	...	Adventure	0.85
13861321	Soundtrack	Alan Silvestri	...	Adventure	0.85
12207086	Opera	Jules Massenet	...	Role-Playing	0.77
1342879	Opera	Giuseppe Verdi	...	Sports	0.70
7675727	Opera	Plácido Domingo	...	Misc	0.50
4591241	Movie	Judy Kuhn	...	Misc	0.50
21097211	Blues	The Blues Brothers	...	Strategy	0.43
13858042	Soundtrack	Alan Silvestri	...	Adventure	0.85

[11 rows x 11 columns]

The features I have created and am using from the datasets are engagement_score emotional mapping. Engagement is created from the ranking in the video games dataset and popularity scores of the songs. Emotional mapping uses valence, tempo, and energy values from the songs to match with genre values from video game data. I manually created a mapping that assigned values to each genre, and was able to merge the song and video game datasets on that.

```
genre_emotion_map = {
  'Strategy': {'energy': .2, 'tempo': .4, 'valence': .7},
  'Sports': {'energy': .9, 'tempo': .8, 'valence': .4},
  'Simulation': {'energy': .6, 'tempo': .5, 'valence': .4},
  'Shooter': {'energy': .6, 'tempo': .4, 'valence': .9},
  'Role-Playing': {'energy': .7, 'tempo': .8, 'valence': .8},
  'Racing': {'energy': .75, 'tempo': .6, 'valence': .4},
  'Puzzle': {'energy': .1, 'tempo': .1, 'valence': .1},
  'Platform': {'energy': .3, 'tempo': .2, 'valence': .3},
  'Misc': {'energy': .5, 'tempo': .5, 'valence': .5},
  'Fighting': {'energy': .8, 'tempo': .8, 'valence': .8},
  'Adventure': {'energy': .85, 'tempo': .9, 'valence': .8},
  'Action': {'energy': 1.0, 'tempo': 1.0, 'valence': .75}}
```

Figure representing the mapping between video game genre and music metadata.

Span refers to the year of release, but with a 10 year consideration buffer. Essentially, if the song or video game came out in 1995, the songs or video games considered for matching have a release year 1990 to 2000. These features are extracted from the old datasets and are used to calculate videogame-song matching.

Categorical Variable Encoding

When training machine learning models, non numerical data must be encoded into numerical values. In order to prepare the data for training, I had to use label encoding on artist and track names. There were about 10 thousand entries in the song track dataset, so one hot encoding took too long. However, one hot encoding worked on the Game Title and genre. Once I encoded this information, I was able to extract a new feature from the data called Engagement_score, and use my other created feature emotional_score to prepare the data for training.

Feature Selection

Feature Importance Evaluation

When evaluating feature importance, I already had domain knowledge. I approached the data with features in mind that I wanted to extract and utilise in my similarity calculations. If I did not know what features would appear most frequently, or were most crucial to the project, I would have used either correlation analysis or a chi square test to determine feature importance. I also learned a lot about RFE feature importance, where the class recursively reduces features until only the desired number of most important features remain. However, since I had domain knowledge, I decided not to write any code calculating the importance of features. I was unsure on this approach until I read up on different approaches to Data Science problems and projects. Overall, the features I chose ended up being good fits for the task at hand.

Dimensionality Reduction

The datasets I chose were large, but not too large or over complicated to the point where it affected computation time or interpretation. I dropped columns I deemed unimportant and merged features or created new ones. Every code I ran on the datasets ran in a few seconds and produced clear results after the preprocessing. Since further dimensionality reduction was not needed for my

chosen data, I did not need to implement specific dimensionality reduction, though I did research PCA and other dimensionality reduction techniques. However, after running the models, it may appear PCA will be required for fine tuning the models.

Data Modeling

Data Splitting

In order to split the data for training the machine learning models, I used a standard 80% training and 20% testing split. I used SKLearn's `train_test_split` function to split the data into the two sets. I opted for an 80/20 split because I wanted to avoid overfitting, which is why I did not use another standard split of 85/15. I also am in the process of manually creating a validation set of data, but this has not been finished by the project due date. I decided to create a validation set that is close in size in order to get experience manually creating a validation set, and to fine tune the machine learning models for milestone 3.

Model Training and Selection

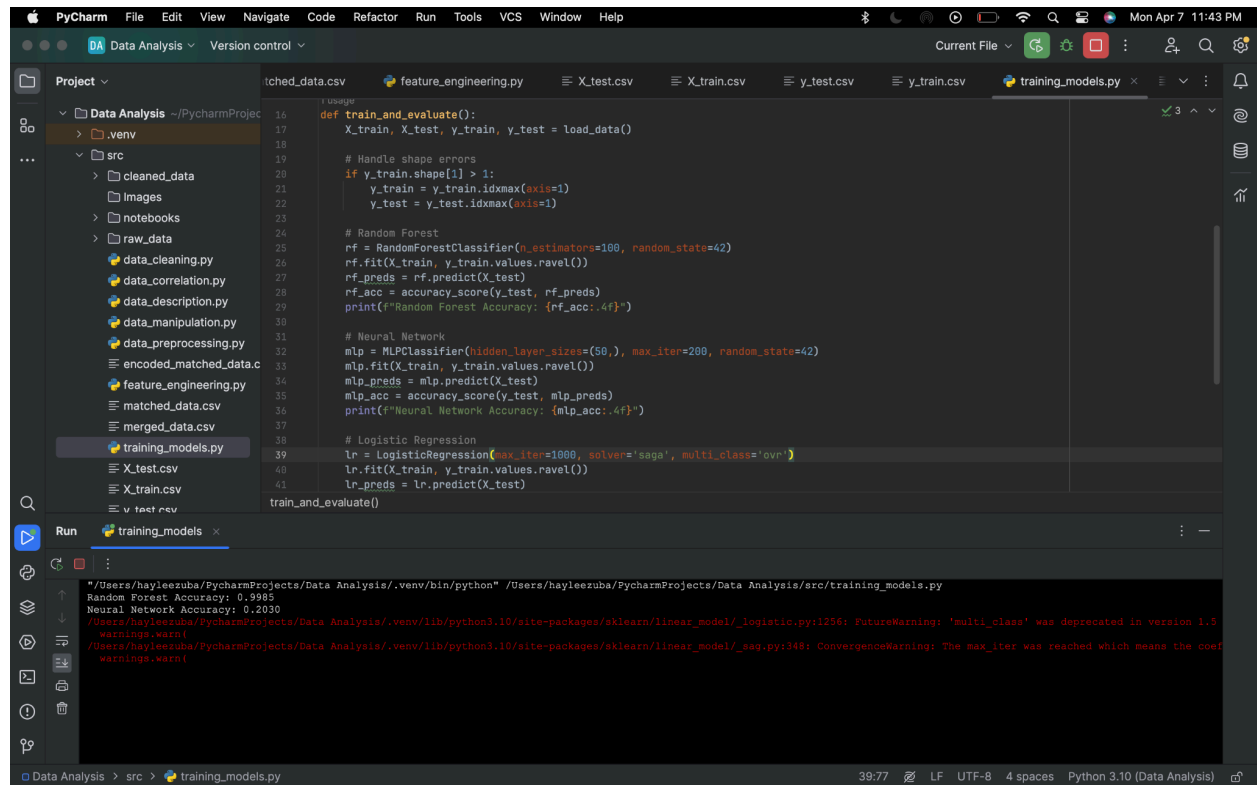
I chose these three models to train: Random Forest, Logistic Regression, and Neural Network. I chose these methods after researching them for various reasons. Random forest is a model that combines multiple decision trees, making

it a very robust model. This model is good to combat overfitting, and works well with categorical data. This model fit my video game and song datasets well because it handles complex data well and provided accurate results. Logistic regression was my next choice because I wanted to really examine if the models would overfit or not. It is fast and efficient and scales well for large datasets. Logistic regression took the longest to run, due to its linear nature. Neural networks is another example of fast, efficient, and scalable models. Neural networks are commonly implemented when nonlinear, complex data is used with little feature engineering.

Model Analysis

After training and testing each model, and using SKLearns built in accuracy checker, these are the results outputted from each model. It appears the Random Forest model is extremely accurate, and the neural network is not accurate due to some errors in formatting. I will have to go in and further explore the data and see where I went wrong and reevaluate if this is a good machine learning model for

the task at hand.



```
16 def train_and_evaluate():
17     X_train, X_test, y_train, y_test = load_data()
18
19     # Handle shape errors
20     if y_train.shape[1] > 1:
21         y_train = y_train.argmax(axis=1)
22         y_test = y_test.argmax(axis=1)
23
24     # Random Forest
25     rf = RandomForestClassifier(n_estimators=100, random_state=42)
26     rf.fit(X_train, y_train.values.ravel())
27     rf_preds = rf.predict(X_test)
28     rf_acc = accuracy_score(y_test, rf_preds)
29     print(f"Random Forest Accuracy: {rf_acc:.4f}")
30
31     # Neural Network
32     mlp = MLPClassifier(hidden_layer_sizes=(50,), max_iter=200, random_state=42)
33     mlp.fit(X_train, y_train.values.ravel())
34     mlp_preds = mlp.predict(X_test)
35     mlp_acc = accuracy_score(y_test, mlp_preds)
36     print(f"Neural Network Accuracy: {mlp_acc:.4f}")
37
38     # Logistic Regression
39     lr = LogisticRegression(max_iter=1000, solver='saga', multi_class='ovr')
40     lr.fit(X_train, y_train.values.ravel())
41     lr_preds = lr.predict(X_test)
42
43     train_and_evaluate()
```

Run training_models

```
"/Users/hayleezuba/PycharmProjects/Data Analysis/.venv/bin/python" /Users/hayleezuba/PycharmProjects/Data Analysis/src/training_models.py
Random Forest Accuracy: 0.9985
Neural Network Accuracy: 0.2030
/Users/hayleezuba/PycharmProjects/Data Analysis/.venv/lib/python3.10/site-packages/sklearn/linear_model/_logistic.py:1256: FutureWarning: 'multi_class' was deprecated in version 1.5
  warnings.warn(
/Users/hayleezuba/PycharmProjects/Data Analysis/.venv/lib/python3.10/site-packages/sklearn/linear_model/_sag.py:348: ConvergenceWarning: The max_iter was reached which means the coef
warnings.warn(
```

The logistic regression model is taking significantly longer to run and evaluate, so I should probably use a different approach. Overall, it seems like the random forest was the best choice model, returning almost 100% accuracy.

Model Evaluation and Interpretation

Model Evaluation

For Milestone 2, I trained 3 machine learning models to predict video games based on song input using training data. The training data was a preprocessed set containing song and video game matchings based on the emotional scores and engagement scores I calculated previously. I briefly evaluated the models performance when matching the test data based on the

Sklearn metrics built in accuracy, F1, and precision scores. Here is what those evaluation scores looked like:

Model	Accuracy	F1- Score	Precision
Random Forest	99.85%	0.9982	0.9980
Linear SVM	75.04%	0.0235	0.0155
Neural Network	20.30%	0.1259	0.1219

Figure 1: Model Evaluation with Accuracy, F1, and precision scores

As we can see, the Random Forest model outperformed Linear SVM and Neural network by a significantly higher percentage. This is most likely due to the ensemble structure of random forest that was highly capable of managing the complexity of the given task. The Linear SVM, in this scenario, acted as a baseline for average model performance. However, Neural Networks incredibly underperformed. This is a reasonable performance expectation, because neural networks are not as capable of handling dimensional data such as the dataset used.

Model Interpretation

Each of the models demonstrated different behaviors when mapping emotional and engagement scores from songs to video games. The Random Forest model consistently performed the best across the three metrics. The ensemble method nature of Random Forest was sufficient in evaluating the complex relationships between features. For that reason, Random Forest is the best model to use for this project. On the other hand, linear SVM scored relatively low on F1 and precision, despite showing moderately high accuracy. This suggests that the model might have defaulted to predicting the majority class, scoring decent accuracy due to class imbalance but failing to meaningfully capture minority class distinctions. Compared to the Random Forest and SVM, the Neural Network performed worse in accuracy, but was more precise than the linear SVM. Overall, this model performed the worst. I believe this is due to the fact that neural networks are sensitive to data dimensionality. So, mapping of emotional scores and engagement scores was most likely too high dimensional for the neural

network to handle. Overall, the models indicated that emotional and engagement scores serve as effective features for genre mapping. However, it is important to consider class imbalances and model selection to properly evaluate the nature of subjective fields - such as personalized recommendations.

Biases and Limitations

The data used to train the models was somewhat limited - I was unable to match every single song in existence with every single video game in existence. So the models are biased and limited to what the dataset offers. Firstly, I'd like to examine some limitations with the given dataset. I already mentioned that not every song and video game that exists can be evaluated by the models. This also means that not every genre of song/video game is listed in the dataset, so the predominantly available data covers mainstream music and video games. This means the data is limited to popular songs and video games, and that's not always the answer. If a person who liked more unique video games/music used the dashboard - they may be disappointed by the results. Ideally, if I made this project to a larger scale I would have incorporated the google search function. I was not able to get that working by the due date, but in a perfect world the model would examine previously matched data, and search the internet for information to calculate the emotional and engagement scores, and match it to a video game.

These limitations are directly related to the biases of the model. For example, within the Hidden Trends in Video Games dataset, the most commonly occurring game genre is Sports at 16%. The next most common genre is Action at 14%. This essentially means there are more sporty video games within the dataset than action, and any other video game genres. The imbalance in data means the model may favor sports recommendations over less-represented genres - like puzzle, strategy, or rpg games. These data imbalances exist for each column in each dataset, no data is 100% equally represented. This is mirroring of real life data, where two categories of data are rarely represented equally.

Tool Development

For this project, I believed a dashboard would be the best outlet to display the findings and interact with the data. I wanted to get creative and display the cross domain project in a fun way. I used an HTML file, because streamlit was causing issues on my computer. The UI design is inspired by many video games and album covers conglomerating into one aesthetically pleasing - if not excessive - interface. Many of my favorite video games and album covers are purple or incorporate purple into their main color scheme. The dashboard features several purple orbs that float around and elastically collide with each other and the sides of the screen. The main purple orb in the center has a text box, prompting the user to input a song. The dashboard will then show a video game that the machine learning model selects for the user. If the user selects a song that is not in the database, a song not found message will pop up, and prompt the user to try a different song. The surrounding orbs will showcase different songs, giving the user an idea of what kind of songs are available from the dataset. When a song is successfully chosen and matched, the match will appear on the orb with accuracy metrics displayed underneath.

Additional Comments:

- The trained models were too large to upload to github, instead saved predictions as a json file
- Presentation slides were modified to explain more about the features, and present clearer EDA results

References:

1. Sci-kit Learn documentation: <http://scikit-learn.org/stable/index.html>
2. Medium Articles:
 - a. <https://medium.com/geekculture/random-forest-ensemble-method-860aaf4fcd16>
 - b. <http://medium.com/@anurag-jain/tf-idf-vectorization-with-cosine-similarity-eca3386d4423>
 - c. <https://medium.com/data-science/how-to-bin-numerical-data-with-pandas-fe5146c9dc55>
 - d. <https://viniciusarz.medium.com/mastering-correlation-analysis-for-data-science-pearson-vs-spearman-cb1cc0a7b456>

- e. <https://medium.com/data-science/feature-selection-with-pandas-e3690ad8504b>

Discovering Hidden Trends in Global Video games, Andy Bramwell, Available:
<https://www.kaggle.com/datasets/thedevastator/discovering-hidden-trends-in-global-video-games>

License: not listed

Audio Features Dataset, available:

<https://www.kaggle.com/datasets/uciml/msd-audio-features/data> Original dataset:

Thierry Bertin-Mahieux, Daniel P.W. Ellis, Brian Whitman, and Paul Lamere. The Million Song Dataset. In Proceedings of the 12th International Society for Music Information Retrieval Conference (ISMIR 2011), 2

License: Unavailable

Spotify Track Features, available:

<https://www.kaggle.com/datasets/nanditapore/spotify-track-features/data>

License: Apache License, Version 2.0

LLM Declarations:

I am not the best with frontend/visual aesthetics using code. I used chatGPT to help me create a visually aesthetic dashboard to meet my lofty aspirations for the project. Here are the prompts I used to assist me in the process:

“If I want to create a dashboard using HTML that displays one main purple circle, 5 surrounding smaller circles, and 3 even smaller circles, how can I go about this? Describe the process to me without showing too much code.”

AI response: “Step-by-Step Plan to Create Your Interactive HTML Dashboard

1. Create the HTML file
2. Design the Circle Layout
3. Make It Interactive
4. Handle Circle Movement and Collisions (Optional)”, with information on how to achieve those tasks. I used this step by step process to create the dashboard on my own.

I also wanted to find a good way to explain why the linear SVM model was moderately accurate, yet super low scoring on F1 and Precision metrics.

Prompt: Why would a linear SVM model perform somewhat accurately, yet score low on F1 and Precision metrics?

AI Response:

“The model is basically **guessing the most common class label** for everything — because it either:

- Can’t learn enough from the data
- Is overwhelmed by **class imbalance**
- Or is underfit and defaulting to the "safest" guess”

I then researched more on predicting the majority class, and found that this is most likely what the model is doing.